Challenge 02: Pincode

Category: Reversing Points: 10 Status: SOLVED

Flag

```
MOBISEC{local_checks_can_be_very_bad_for_security} (PIN 703958)
```

Source: Typed into the CTFd flag box; solve confirmed live from /user

Description / Methodology

Brute-forced all 1,000,000 six-digit PINs against 10,000 rounds of MD5 each, using 4-core multiprocessing (~1h50m runtime), then submitted the recovered PIN to the server.

The PIN check runs entirely on-device with no rate limiting -- all 1,000,000 combinations can be brute-forced offline.

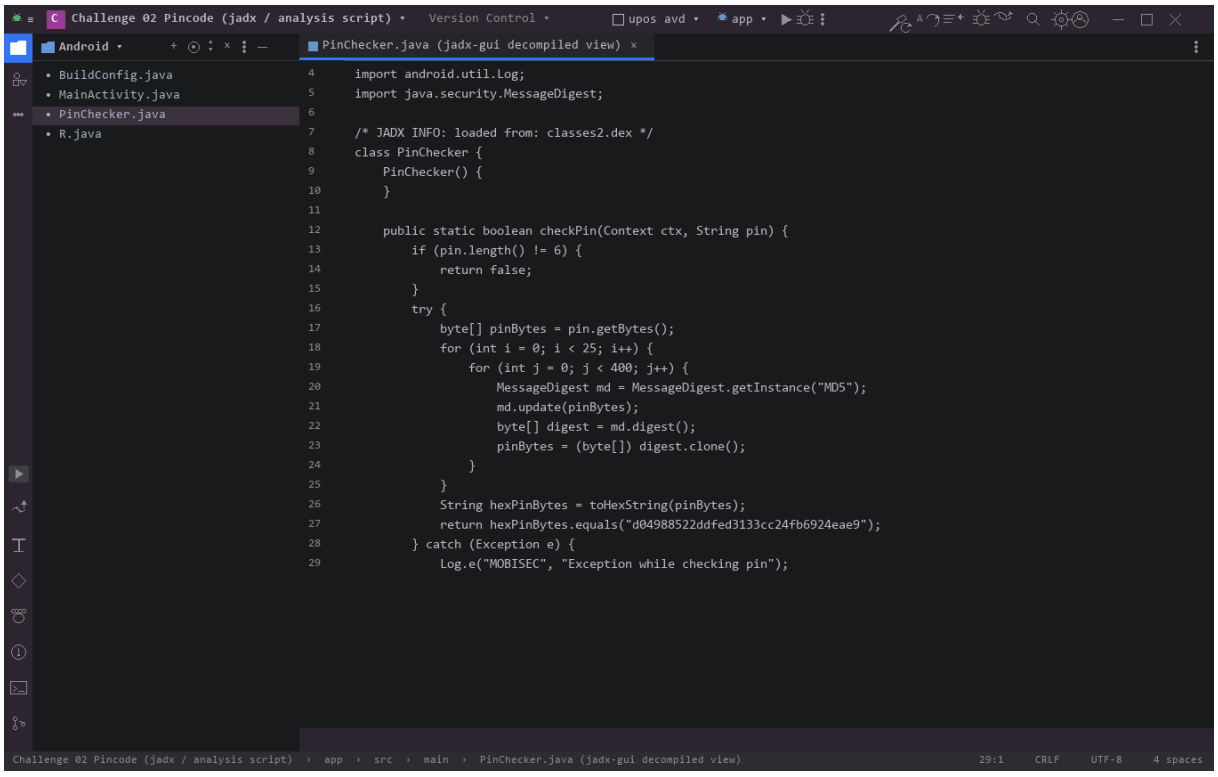
How I Got The Flag

1. I decompiled the APK and found the PIN check runs entirely on-device, hashing the candidate PIN through 10,000 rounds of MD5 with no rate limiting.

2. I wrote a brute-force script using 4-core multiprocessing to try all 1,000,000 six-digit PINs against the target digest (about 1h50m runtime).

3. Once the correct PIN was found, I typed it into the flag box to get the flag.

Project Layout



Exact Code (from disk)

PinChecker.java (jadx decompiled)

```
package com.mobisec.pincode;
import android.content.Context;
import android.util.Log;
import java.security.MessageDigest;
/* JADX INFO: loaded from: classes2.dex */
class PinChecker {
    PinChecker() {
    }
    public static boolean checkPin(Context ctx, String pin) {
        if (pin.length() != 6) {
            return false;
        }
        try {
            byte[] pinBytes = pin.getBytes();
            for (int i = 0; i < 25; i++) {
                for (int j = 0; j < 400; j++) {
                    MessageDigest md = MessageDigest.getInstance("MD5");
                    md.update(pinBytes);
                    byte[] digest = md.digest();
                    pinBytes = (byte[]) digest.clone();
                }
            }
            String hexPinBytes = toHexString(pinBytes);
            return hexPinBytes.equals("d04988522ddfed3133cc24fb6924eae9");
        } catch (Exception e) {
            Log.e("MOBISEC", "Exception while checking pin");
            return false;
        }
    }
    public static String toHexString(byte[] bytes) {
        StringBuilder hexString = new StringBuilder();
        for (byte b : bytes) {
            String hex = Integer.toHexString(b & 255);
            if (hex.length() == 1) {
                hexString.append('0');
            }
            hexString.append(hex);
        }
        return hexString.toString();
    }
}
```

brute.py

```
import hashlib
import multiprocessing as mp
TARGET = "d04988522ddfed3133cc24fb6924eae9"
def check(pin):
    b = pin.encode()
    for _ in range(25):
        for _ in range(400):
            md = hashlib.md5()
            md.update(b)
            b = md.digest()
    return b.hex() == TARGET
def worker(rng):
    lo, hi = rng
    for i in range(lo, hi):
```

```
        pin = "%06d" % i
        if check(pin):
            return pin
    return None
def main():
    n = mp.cpu_count()
    chunk = 1000000 // n + 1
    ranges = [(i, min(i + chunk, 1000000)) for i in range(0, 1000000, chunk)]
    with mp.Pool(n) as p:
        for res in p.imap_unordered(worker, ranges):
            if res:
                print("FOUND:", res)
                p.terminate()
                return
    print("not found")
if __name__ == "__main__":
    main()
```